Desafio Técnico: Sistema de Cadastro de Clientes

Documento de requisitos · versão 1.0 · Oct 2, 2026 · @Urtium

1. Objetivo e formato

Uma dupla de desenvolvedores júnior constrói, em 10 dias úteis, um cadastro de clientes completo: API REST e interface web. O trabalho segue o fluxo de um time real, e a avaliação mede tanto o produto quanto o caminho até ele (planejamento, Gitflow, revisão de código e releases).

- **Equipe:** os dois devs trabalham no mesmo repositório GitHub; os avaliadores entram como colaboradores.
- **Prazo sugerido:** 10 dias úteis até a release v1.0.0, mais 1 dia útil para um hotfix pedido pelos avaliadores (seção 13). Ajustável.
- **Entrega:** tag v1.0.0 na branch main e apresentação de 30 minutos (seção 16).
- **Dúvidas:** abertas como issue no repositório, com a label `question`, para ficarem registradas junto do trabalho.
- **Idioma:** código, commits e nomes de branch em inglês; interface, README e textos da documentação da API em português.

2. Requisitos obrigatórios

Os 15 itens abaixo são o mínimo exigido; faltar qualquer um conta como entrega incompleta. Cada um tem critérios de aceite na seção indicada.

#	Requisito	Detalhado em
1	Node.js com TypeScript	Seções 5 e 10
2	Prisma como ORM	Seções 6 e 11
3	Técnicas de tipagem forte	Seção 10
4	Ao menos uma lib type safe na API (recomendada: Zod com fastify-type-provider-zod)	Seções 5 e 8

#	Requisito	Detalhado em
5	API REST	Seção 8
6	Interface em React, Vite e Tailwind CSS: listagem, wizard de cadastro, edição, soft delete e exclusão definitiva	Seção 9
7	Interface funcionando no celular	Seção 9
8	PostgreSQL leve com dados que sobrevivem a reinícios	Seção 11
9	Migrations versionadas	Seção 11
10	Gitflow	Seção 13
11	Clean code	Seção 14
12	Sistema conteinerizado	Seção 12
13	Inicialização com um único comando, via shell script	Seção 12
14	Tela para restaurar clientes excluídos (lixeira)	Seção 9
15	Documentação da API	Seção 8

3. Escopo

O sistema cobre o ciclo de vida inteiro de um cliente: cadastro, edição, lixeira, restauração e exclusão definitiva. Não há login.

Dentro do escopo

- API REST de clientes: listar, detalhar, cadastrar, editar, mover para a lixeira, restaurar e excluir definitivamente.
- Interface web responsiva para todos esses fluxos.
- Banco PostgreSQL com migrations e dados persistentes.
- Containers para banco, API e interface, iniciados por um único script.
- Documentação interativa da API.

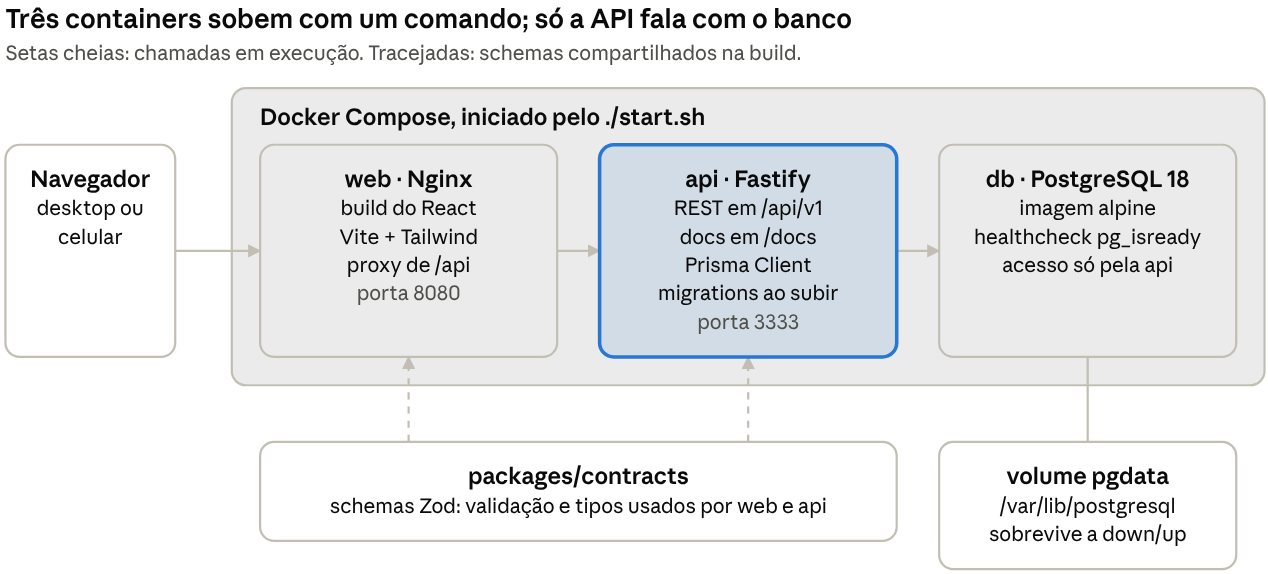
Fora do escopo

- Login, perfis e permissões.

- Deploy em nuvem.
- Vários endereços por cliente, anexos e histórico de alterações.
- Outros idiomas na interface.

4. Arquitetura e estrutura do repositório

O sistema roda em três containers (interface, API e banco), e o código vive em um monorepo com um pacote de contratos compartilhado.



arquitetura · 3 containers, 1 pacote compartilhado, 1 volume

O navegador só conversa com o Nginx, que repassa `/api` para a API; só a API acessa o banco. O pacote de contratos não é um container: é código importado por web e api na build.

Estrutura do repositório (npm workspaces):

```
cadastro-clientes/
  apps/
    api/            Fastify + Prisma
    web/            React + Vite + Tailwind
  packages/
    contracts/      schemas Zod e tipos compartilhados
  docker-compose.yml
  start.sh
  .env.example
  README.md
```

5. Stack

A lib type safe recomendada é o Zod integrado ao Fastify via fastify-type-provider-zod. O mesmo schema valida a requisição, gera os tipos da rota e alimenta o OpenAPI, então código, tipos e documentação não se desencontram.

Camada	Obrigatório	Recomendado
Linguagem	Node.js LTS e TypeScript em modo estrito	Versão do Node fixada em `.nvmrc` e no Dockerfile
API	REST, Zod e uma lib de rotas type safe	Fastify com fastify-type-provider-zod
ORM e migrations	Prisma e Prisma Migrate	Prisma Studio para inspeção local
Banco	PostgreSQL na imagem oficial alpine	Tag de versão fixa, nunca `latest`
Documentação da API	OpenAPI 3 gerado a partir dos schemas	@fastify/swagger com Swagger UI ou Scalar em `/docs`
Interface	React, Vite e Tailwind CSS	React Router, TanStack Query e React Hook Form
Formulários	Validação com os schemas Zod compartilhados	@hookform/resolvers (zodResolver)
Componentes	Diálogos e menus acessíveis por teclado	Radix UI ou Headless UI

Camada	Obrigatório	Recomendado
Infra	Docker, Docker Compose e `start.sh`	Nginx servindo o build da interface e fazendo proxy de `/api`
Qualidade	Lint e formatação sem erros nem avisos	typescript-eslint (strict-type-checked) com Prettier, ou Biome
Monorepo	Um repositório com API, interface e contratos	npm workspaces

Alternativa aceita para a API: Hono com @hono/zod-openapi, desde que entregue as mesmas garantias. tRPC não atende, porque não é REST.

6. Modelo de dados

São duas tabelas: `customers` (cliente PF ou PJ) e `addresses` (um endereço por cliente, apagado junto na exclusão definitiva). No banco, tabelas e colunas ficam em snake_case (com `@@map` e `@map`); no código, em camelCase.

Customer

Campo	Tipo	Obrigatório	Regra
id	UUID	Sim	Gerado no cadastro
personType	enum INDIVIDUAL ou COMPANY	Sim	PF ou PJ; não muda depois do cadastro
document	texto	Sim	CPF (11) ou CNPJ (14) sem máscara; único em toda a base; não muda
name	texto, 3 a 150 caracteres	Sim	Nome completo (PF) ou razão social (PJ)
tradeName	texto, até 150 caracteres	Só PJ, opcional	Nome fantasia
birthDate	data	Só PF, obrigatório	Não pode ser futura nem anterior a 1900
email	texto, até 254 caracteres	Sim	Formato válido, salvo em minúsculas

Campo	Tipo	Obrigatório	Regra
phone	texto	Sim	10 ou 11 dígitos, com DDD
createdAt	data e hora	Sim	Preenchido pelo banco
updatedAt	data e hora	Sim	Atualizado a cada alteração
deletedAt	data e hora	Não	Preenchido no soft delete; nulo enquanto ativo

Address

Campo	Tipo	Obrigatório	Regra
id	UUID	Sim	Gerado no cadastro
customerId	UUID	Sim	Chave estrangeira única para customers, com exclusão em cascata
zipCode	texto	Sim	CEP com 8 dígitos
street	texto	Sim	Logradouro, até 150 caracteres
number	texto	Sim	Até 10 caracteres; aceita "S/N"
complement	texto	Não	Até 100 caracteres
district	texto	Sim	Bairro, até 100 caracteres
city	texto	Sim	Até 100 caracteres
state	texto, 2 caracteres	Sim	Uma das 27 UFs

Índices obrigatórios: único em `customers.document`, comum em `customers.deleted_at` e único em `addresses.customer_id`.

No banco, PF e PJ dividem a mesma tabela com colunas opcionais. Na API e na interface, o cliente é uma união discriminada por `personType`, e a conversão entre os dois formatos fica em um mapper (seção 10).

7. Regras de negócio

O documento (CPF ou CNPJ) identifica o cliente: é único em toda a base, validado pelos dígitos verificadores e nunca muda. As demais regras definem o ciclo ativo, lixeira e exclusão definitiva.

Código	Regra	Resposta da API
RN01	Documento único em toda a base, inclusive na lixeira, garantido por constraint no banco (não só por validação na aplicação)	Ativo: 409 `customer-document-conflict`. Na lixeira: 409 `customer-in-trash`, com o `customerId`
RN02	CPF com 11 dígitos, dígitos verificadores corretos e sem sequência repetida (como 111.111.111-11)	400 no campo `document`
RN03	CNPJ com 14 caracteres e dígitos verificadores corretos, aceitando o formato alfanumérico (detalhes abaixo)	400 no campo `document`
RN04	`personType` e `document` não mudam depois do cadastro	400 se vierem na edição
RN05	Campo exclusivo de um tipo não vale para o outro: `birthDate` só PF, `tradeName` só PJ	400 no campo enviado
RN06	Documento, telefone e CEP chegam com ou sem máscara e são salvos sem pontuação; textos passam por trim; e-mail vai para minúsculas e letras do CNPJ para maiúsculas	Respostas trazem sempre o valor normalizado
RN07	Soft delete preenche `deletedAt`: o cliente sai da listagem, não pode ser editado e aparece na lixeira	Editar ou mover para a lixeira de novo: 404
RN08	Restaurar limpa `deletedAt` e devolve o cliente à listagem com os dados originais	Cliente fora da lixeira: 409

Código	Regra	Resposta da API
RN09	Exclusão definitiva apaga cliente e endereço, esteja ativo ou na lixeira, e não pode ser desfeita	Cliente inexistente: 404
RN10	Ordem padrão: listagem por `createdAt` decrescente; lixeira por `deletedAt` decrescente	Alterável por `sortBy` e `sortOrder`
RN11	Datas trafegam em ISO 8601 (UTC para data e hora, AAAA-MM-DD para `birthDate`) e aparecem como dd/mm/aaaa na interface	Formato inválido: 400

CNPJ alfanumérico. A partir de julho de 2026, novos CNPJs podem ter letras (A a Z) nas 12 primeiras posições; os 2 dígitos verificadores continuam numéricos e a máscara não muda. O cálculo segue o módulo 11, com cada caractere valendo seu código ASCII menos 48 (A = 17, B = 18 e assim por diante). Fonte: perguntas e respostas da Receita Federal. CNPJs fictícios para teste podem ser gerados no Simulador Nacional de CNPJ.

Casos de teste mínimos dos validadores

Valor	Resultado esperado
529.982.247-25	CPF válido
529.982.247-26	CPF inválido (dígito verificador)
111.111.111-11	CPF inválido (sequência repetida)
11.222.333/0001-81	CNPJ numérico válido
11.222.333/0001-82	CNPJ inválido (dígito verificador)
12.ABC.345/01DE-35	CNPJ alfanumérico válido (exemplo oficial da Receita)
12abc34501de35	CNPJ alfanumérico válido depois de normalizado

8. API REST e documentação

A API tem 8 rotas de clientes sob `/api/v1`, mais um healthcheck. Erros seguem o padrão Problem Details (RFC 9457) e a documentação interativa fica em `/docs`.

Convenções

- JSON em UTF-8; ids em UUID (id mal formado: 400).
- Cadastro responde 201 com o header `Location` apontando para o novo cliente.
- Erros em `application/problem+json` (RFC 9457), com a lista de campos inválidos quando houver.
- Erro 500 nunca expõe stack trace nem mensagem do Prisma; o detalhe vai para o log.
- Listagens paginadas no servidor: `page` (padrão 1), `pageSize` (padrão 10, máximo 50), `search` (nome, e-mail ou documento, sem diferenciar maiúsculas), `sortBy` e `sortOrder`.
- Listagens devolvem os itens em `data` e a paginação em `meta` (`page`, `pageSize`, `total`, `totalPages`).
- A interface chama a API por caminho relativo (`/api`): proxy do Nginx no container e do Vite em desenvolvimento, sem depender de CORS.

Rotas

Método	Rota	O que faz	Sucesso	Erros
GET	`/health`	Confere a API e a conexão com o banco	200	503
GET	`/api/v1/customers`	Lista clientes ativos	200	400
GET	`/api/v1/customers/deleted`	Lista clientes na lixeira	200	400
GET	`/api/v1/customers/{id}`	Detalha um cliente ativo	200	400, 404
POST	`/api/v1/customers`	Cadastra um cliente	201	400, 409
PATCH	`/api/v1/customers/{id}`	Edita um cliente ativo	200	400, 404

Método	Rota	O que faz	Sucesso	Erros
DELETE	`/api/v1/customers/{id}`	Move para a lixeira (soft delete)	204	404
POST	`/api/v1/customers/{id}/restore`	Restaura da lixeira	200	404, 409
DELETE	`/api/v1/customers/{id}/permanent`	Exclui definitivamente	204	404

Exemplos

Cadastro de PF (`POST /api/v1/customers`). Para PJ, `personType` é COMPANY, o CNPJ vai em `document`, `tradeName` é opcional e não existe `birthDate`.

```
{
  "personType": "INDIVIDUAL",
  "name": "Maria da Silva",
  "document": "529.982.247-25",
  "birthDate": "1990-05-14",
  "email": "Maria.Silva@Exemplo.com.br",
  "phone": "(11) 98765-4321",
  "address": {
    "zipCode": "01310-100",
    "street": "Avenida Paulista",
    "number": "1000",
    "complement": "Apto 12",
    "district": "Bela Vista",
    "city": "São Paulo",
    "state": "SP"
  }
}
```

Resposta 201: o cliente completo, já normalizado.

```
{
  "id": "6f1c2b9e-3d4a-4c8b-9a7e-1f2d3c4b5a69",
  "personType": "INDIVIDUAL",
  "name": "Maria da Silva",
  "document": "52998224725",
  "birthDate": "1990-05-14",
  "email": "maria.silva@exemplo.com.br",
```

```
  "phone": "11987654321",
  "address": {
    "zipCode": "01310100",
    "street": "Avenida Paulista",
    "number": "1000",
    "complement": "Apto 12",
    "district": "Bela Vista",
    "city": "São Paulo",
    "state": "SP"
  },
  "createdAt": "2026-10-05T13:45:00.000Z",
  "updatedAt": "2026-10-05T13:45:00.000Z",
  "deletedAt": null
}
```

Erro de validação (400):

```
{
  "type": "/problems/validation-error",
  "title": "Dados inválidos",
  "status": 400,
  "detail": "Um ou mais campos não passaram na validação.",
  "instance": "/api/v1/customers",
  "errors": [
    { "field": "document", "message": "CPF inválido" },
    { "field": "address.zipCode", "message": "CEP deve ter 8 dígitos" }
  ]
}
```

Documento de um cliente que está na lixeira (409):

```
{
  "type": "/problems/customer-in-trash",
  "title": "Cliente na lixeira",
  "status": 409,
  "detail": "Já existe um cliente com este documento na lixeira.",
  "instance": "/api/v1/customers",
  "customerId": "6f1c2b9e-3d4a-4c8b-9a7e-1f2d3c4b5a69"
}
```

Edição (`PATCH`): campos omitidos ficam como estão e `address` aceita atualização parcial.

```
{
  "phone": "11912345678",
  "address": { "number": "1200" }
}
```

Documentação

- OpenAPI 3 gerado automaticamente a partir dos schemas Zod; especificação escrita à mão não é aceita.
- UI interativa em `http://localhost:3333/docs`, com a especificação em JSON disponível para download.
- Toda rota com resumo, descrição, tag, parâmetros, corpo, todas as respostas possíveis (inclusive erros) e exemplos.
- Aceite: com o sistema no ar via `start.sh`, todos os endpoints podem ser executados pela própria UI de documentação.

9. Interface web

A interface tem 4 telas, funciona do celular ao desktop e trata carregamento, vazio e erro em todas elas. Textos em português, com mensagens de erro junto ao campo.

Telas e rotas

Rota	Tela	Conteúdo
`/`	Clientes	Listagem de ativos com busca, ordenação, paginação e ações
`/clientes/novo`	Novo cliente	Wizard de cadastro em 4 etapas
`/clientes/:id/editar`	Editar cliente	Formulário com os dados atuais
`/lixeira`	Lixeira	Clientes excluídos, com restaurar e excluir definitivamente
Qualquer outra	Não encontrado	Página 404 com link para a listagem

Listagem (página principal)

- Tabela no desktop; abaixo de 768 px (breakpoint `md` do Tailwind), cada cliente vira um card.
- Mostra nome (com nome fantasia abaixo, para PJ), tipo, documento e telefone formatados, e-mail, cidade/UF e data de cadastro.
- Busca com debounce de 300 ms por nome, e-mail ou documento, com ou sem máscara.
- Página, busca e ordenação ficam na URL: recarregar ou compartilhar o link mantém o estado.
- Ações por cliente: Editar, Mover para a lixeira e Excluir definitivamente. Botão "Novo cliente" sempre visível.
- Estados: carregando (skeleton), lista vazia (com atalho para cadastrar), busca sem resultado e erro (com "Tentar novamente").

Wizard de cadastro

1. **Identificação:** Pessoa Física ou Pessoa Jurídica; os campos mudam conforme o tipo (PF: nome completo, CPF e data de nascimento; PJ: razão social, nome fantasia e CNPJ).
2. **Contato:** e-mail e telefone.
3. **Endereço:** CEP, logradouro, número, complemento, bairro, cidade e UF (lista com as 27 UFs).
4. **Revisão:** resumo de tudo, com atalho para editar cada bloco, e o botão "Cadastrar".

Regras do wizard:

- Indicador de progresso ("Etapa 2 de 4").
- Só avança com a etapa válida; erros aparecem junto ao campo, em português.
- Voltar não perde nada. Trocar de PF para PJ limpa só os campos que não valem para o novo tipo.
- Máscaras para CPF, CNPJ (inclusive alfanumérico), telefone e CEP.
- Botão de envio desabilitado durante a requisição, para não cadastrar duas vezes.
- Sucesso: aviso de confirmação (toast) e volta à listagem com o novo cliente visível.
- Erro 409: mensagem no campo documento; se o cliente estiver na lixeira, oferecer "Restaurar cadastro existente".

Edição

- Reaproveita os componentes e as validações do wizard, em página única com seções.
- Tipo de pessoa e documento aparecem bloqueados.
- Salva via `PATCH`; depois, toast e volta à listagem.
- Se o cliente não existir mais (404), avisa e volta à listagem.

Exclusões e lixeira

- **Mover para a lixeira:** diálogo de confirmação simples, depois toast de sucesso.
- **Excluir definitivamente:** diálogo destrutivo que avisa que a ação é irreversível; o botão só habilita depois de digitar EXCLUIR.
- **Lixeira:** clientes excluídos com a data da exclusão, busca e paginação; ações Restaurar e Excluir definitivamente; estado vazio próprio.
- Depois de cada ação, as listas afetadas se atualizam sem recarregar a página.

Celular e acessibilidade

- Mobile first, sem rolagem horizontal da página entre 360 px e 1440 px de largura.
- Validado em 360 × 640, 390 × 844 e 1280 × 800.
- Navegação compacta no celular (menu ou barra inferior).
- Alvos de toque com pelo menos 44 px; no celular, os botões do wizard ficam fixos no rodapé.
- Todo campo com label, foco visível e diálogos operáveis por teclado (foco preso no diálogo, Esc fecha).
- Contraste mínimo WCAG AA.

10. Tipagem forte

Dado inválido não compila ou não passa da fronteira: os schemas Zod são a única fonte dos tipos e o compilador roda no modo mais estrito em todos os pacotes.

- `tsconfig` base compartilhado, com no mínimo estas opções:

```
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noImplicitOverride": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true
  }
}
```

- Proibido no código (lint em modo erro): `any` explícito, `@ts-ignore` e non-null assertion (`!`). Um `@ts-expect-error` inevitável leva justificativa.
- Asserções `as` só em fronteiras, com comentário explicando; `as const` é livre.
- Schemas Zod são a fonte única: os tipos saem de `z.input` e `z.output` (ou `z.infer`). Interface escrita à mão que duplica um schema não é aceita.
- Schemas de entrada rejeitam campos desconhecidos (modo estrito do Zod).
- Cliente modelado como união discriminada por `personType`, na API e na interface, com `switch` exaustivo (verificação `never` no `default`).
- Branded types para `CustomerId`, `Cpf` e `Cnpj`: só valores validados circulam com esses tipos.
- Variáveis de ambiente validadas com Zod na inicialização; valor ausente ou inválido derruba a aplicação com mensagem clara.
- Toda rota declara o schema de resposta de cada status code, e o retorno do handler é checado pelo compilador.
- Tipos do Prisma não saem da camada de repositório: mappers convertem a linha do banco em tipo de domínio.
- Na interface: cliente HTTP tipado (nada de `fetch` devolvendo `any`), respostas validadas com os schemas do pacote de contratos e formulários com zodResolver usando os mesmos schemas da API.
- `tsc --noEmit` passa sem erros em todos os pacotes, e o build falha se houver erro de tipo.

Exemplo ilustrativo do que se espera no pacote de contratos (a sintaxe exata depende da versão do Zod):

```
export const cpfSchema = z.string().transform(onlyDigits).refine(isValidCpf,
'CPF inválido').brand<'Cpf'>();

const individualSchema = z.object({
  personType: z.literal('INDIVIDUAL'),
  name: nameSchema,
  document: cpfSchema,
  birthDate: birthDateSchema,
  ...contactAndAddressShape,
});

const companySchema = z.object({
  personType: z.literal('COMPANY'),
  name: nameSchema,
  tradeName: tradeNameSchema.optional(),
  document: cnpjSchema,
  ...contactAndAddressShape,
});

export const createCustomerSchema = z.discriminatedUnion('personType',
[individualSchema, companySchema]);
export type CreateCustomerInput = z.input<typeof createCustomerSchema>; // o
que o formulário envia
export type CreateCustomer = z.output<typeof createCustomerSchema>; // o que
o service recebe

export function displayName(customer: Customer): string {
  switch (customer.personType) {
    case 'INDIVIDUAL':
      return customer.name;
    case 'COMPANY':
      return customer.tradeName ?? customer.name;
    default:
      return assertNever(customer);
  }
}
```

11. Banco de dados e migrations

O banco é o PostgreSQL 18 na imagem oficial alpine (a variante leve), com os dados em um volume nomeado. Nada se perde ao reiniciar, recriar containers ou rodar o `start.sh` de novo.

Persistência

- Imagem `postgres:18-alpine`, com a tag de versão fixa no `docker-compose.yml`.
- Volume nomeado (ex.: `pgdata`) montado em `/var/lib/postgresql`. A partir da versão 18 esse é o caminho certo; tutoriais antigos montam em `/var/lib/postgresql/data`, e aí os dados vão para um volume anônimo que some quando o container é recriado (documentação da imagem oficial).
- Dados só são apagados por comando explícito (`./start.sh --reset` ou `docker compose down -v`).
- Credenciais em variáveis de ambiente: `.env.example` versionado, `.env` no `.gitignore`, nenhum segredo no código.
- Healthcheck com `pg_isready`; a API só sobe depois do banco saudável.

Migrations

- Prisma Migrate, com as migrations versionadas em `apps/api/prisma/migrations`.
- Em desenvolvimento, `prisma migrate dev`; no container, a API roda `prisma migrate deploy` antes de iniciar o servidor.
- `prisma db push` não entra no fluxo do projeto.
- Migration aplicada não é editada: cada mudança de schema gera uma nova. O histórico precisa mostrar a evolução real do modelo (não vale apagar tudo e gerar uma migration única no fim).
- Busca por nome e e-mail ignora maiúsculas e minúsculas; busca por documento compara o valor normalizado.

Aceite: cadastrar um cliente, rodar `docker compose down`, rodar `./start.sh` de novo e o cliente continuar lá.

12. Containerização e start.sh

Um clone limpo do repositório, numa máquina só com Docker, sobe tudo com `./start.sh`. Sem passos manuais e sem instalar Node na máquina.

Containers

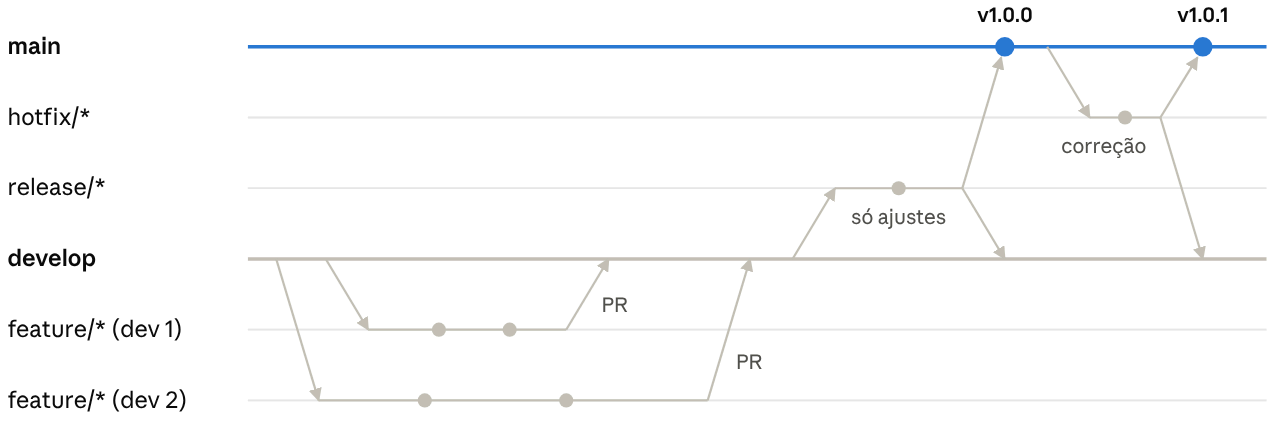
- `docker-compose.yml` com três serviços: `db`, `api` e `web`.
- Dockerfiles multi-stage para `api` e `web`; imagens finais sem dependências de desenvolvimento.
- A imagem da API roda com usuário não root e aplica as migrations antes de subir o servidor.
- `web`: build do Vite servido pelo Nginx, com fallback de SPA (recarregar `/lixeira` não dá 404) e proxy de `/api` para o serviço `api`.
- Ordem de subida garantida por `depends_on` com `condition: service_healthy`; a API expõe `GET /health` para o healthcheck.
- Portas configuráveis no `.env`, com padrão: interface em `http://localhost:8080` e API em `http://localhost:3333`. O banco não precisa ficar exposto ao host.
- No dia a dia, API e interface podem rodar fora do container com hot reload, usando só o banco via Compose. O README explica os dois modos.

start.sh

- Começa com `#!/usr/bin/env bash` e `set -euo pipefail`.
- Confere os pré-requisitos (Docker instalado, daemon rodando, Docker Compose v2) e falha com mensagem clara se faltar algo.
- Cria o `.env` a partir do `.env.example` quando ele não existe.
- Faz build e sobe os serviços em segundo plano (`docker compose up -d --build`).
- Espera a API ficar saudável, com tempo limite. Se estourar, mostra os logs relevantes e sai com código diferente de zero.
- Ao final, imprime os endereços da interface, da API e da documentação.
- É idempotente: rodar duas vezes seguidas não quebra nada nem apaga dados.
- Funciona em Linux, macOS e Windows com WSL2.
- Opções como `--help`, `--stop`, `--logs`, `--seed` e `--reset` (com confirmação) contam como diferencial.

13. Fluxo de trabalho com Gitflow

Todo código chega à `main` por Pull Request revisado pelo outro dev. Features saem de `develop`; releases e hotfixes chegam à `main` com tag de versão e voltam para `develop`.



Gitflow · features em paralelo, release 1.0.0 e hotfix 1.0.1

Cada dev trabalha na própria feature, e o outro revisa o PR. A `main` só recebe código por release ou hotfix, sempre com tag, e tudo volta para `develop`.

Regras

- Commit direto em `main` e `develop` é bloqueado por branch protection no GitHub.
- Nomes de branch: `feature/<issue>-<descricao>` (ex.: `feature/12-customer-wizard`), `release/<versão>` e `hotfix/<versão>`.
- Toda mudança entra por Pull Request, com aprovação obrigatória do outro dev antes do merge.
- PRs pequenos e focados: um por issue, idealmente até cerca de 400 linhas alteradas (sem contar arquivos gerados).
- Template de PR com o que muda, como testar, prints ou vídeo para mudanças de interface e checklist (lint, typecheck, build).
- Revisão de verdade: comentários com sugestões e pedidos de ajuste, não só "aprovado".
- Commits no padrão Conventional Commits (`feat:`, `fix:`, `refactor:`, `docs:`, `test:`, `chore:`, `ci:`, `build:`).
- Release: `release/x.y.z` sai de `develop` só com ajustes finais e CHANGELOG; entra na `main` com tag anotada (`v1.0.0`) e volta para `develop`.

Planejamento

- Backlog no GitHub Projects com as colunas Backlog, A fazer, Em andamento, Em revisão e Concluído.
- Cada requisito vira uma ou mais issues com critério de aceite; o PR fecha a issue (`Closes #12`).
- Os dois devs contribuem nas duas camadas: cada um precisa de ao menos um PR relevante na API e um na interface.

Hotfix obrigatório

Depois da tag v1.0.0, os avaliadores abrem uma issue de bug. A dupla corrige pelo fluxo de hotfix (`hotfix/1.0.1` a partir da `main`, merge em `main` e `develop`, tag `v1.0.1`) em até 1 dia útil.

14. Clean code

Cada camada tem uma responsabilidade só, e qualquer pessoa do time acha onde mora uma regra sem precisar perguntar.

API em camadas

- **routes:** declara rotas, schemas e status codes; não tem regra de negócio nem acesso ao banco.
- **services:** regras de negócio (unicidade, imutabilidade, lixeira, exclusão).
- **repositories:** único lugar que usa o Prisma; devolve tipos de domínio via mappers.
- **errors:** erros de domínio tipados (ex.: `CustomerNotFoundError`, `DocumentConflictError`), convertidos em Problem Details por um error handler central.
- Configuração centralizada (env validado) e logger estruturado (pino, já integrado ao Fastify), sem `console.log`.

Estrutura sugerida:

```
apps/api/src/
  config/env.ts
  modules/customers/
    customer.routes.ts
    customer.service.ts
    customer.repository.ts
    customer.mapper.ts
    customer.errors.ts
  shared/http/error-handler.ts
  app.ts
  server.ts
```

Interface

- Organização por funcionalidade (`features/customers` com `components` , `hooks` , `api` e `pages`); componentes genéricos em `components/ui` .
- Acesso a dados em hooks (TanStack Query), não espalhado pelas telas.
- Componentes pequenos, reaproveitados entre wizard e edição.

Geral

- Nomes descritivos e consistentes; funções curtas, com uma responsabilidade.
- Sem números e textos mágicos: constantes nomeadas (ex.: `MAX_PAGE_SIZE = 50`).
- Sem código morto, comentado ou `console.log` esquecido.
- Lint e formatação sem erros nem avisos em todos os pacotes.
- README com: como rodar, arquitetura, decisões tomadas (e por quê) e o que ficou de fora.

15. Diferenciais

Itens opcionais que somam até 10 pontos extras (seção 16). Só contam se todos os requisitos obrigatórios estiverem atendidos.

- Testes unitários dos validadores e services, e de integração das rotas com banco de teste (Vitest).
- Testes end-to-end do fluxo principal (Playwright).
- CI no GitHub Actions rodando lint, typecheck, testes e build em todo PR, como check obrigatório para o merge.

- Endereço preenchido automaticamente pelo CEP (ViaCEP), com digitação manual se a consulta falhar.
- Toast com "Desfazer" depois de mover para a lixeira.
- Confirmação ao sair do wizard com dados não salvos.
- Controle de concorrência na edição (ETag com `If-Match`, ou `updatedAt`), com 409 ou 412 em caso de conflito.
- Constraint CHECK no banco garantindo a coerência entre tipo de pessoa e campos (SQL na migration).
- `exactOptionalPropertyTypes` habilitado no tsconfig.
- Spec OpenAPI exportada para o repositório e conferida no CI.
- Seed com clientes de exemplo (`./start.sh --seed`) e as demais opções do script.
- ShellCheck sem avisos no `start.sh`.
- Tema escuro.

16. Entrega e avaliação

A nota vai de 0 a 100, mais até 10 pontos de diferenciais, e só é calculada se o sistema subir com `./start.sh` numa máquina limpa. Sem isso, a avaliação técnica para nesse ponto.

Entregáveis

- Repositório no GitHub com acesso para os avaliadores.
- Tags `v1.0.0` e `v1.0.1` na `main`, com CHANGELOG.
- README completo (seção 14).
- Quadro do GitHub Projects com o histórico das issues.
- Apresentação de 30 minutos: demo no desktop e no celular, passeio pelos PRs e decisões, e perguntas. Cada dev explica uma parte que fez e uma parte que o outro fez.

Pesos

Critério	Peso (pontos)	O que é observado
Funcionalidades	20	Requisitos das seções 7 e 9 funcionando, sem bugs nos fluxos principais

Critério	Peso (pontos)	O que é observado
Fluxo de trabalho	15	Gitflow, PRs pequenos, revisões reais, commits, issues, releases e hotfix
Tipagem forte	15	Seção 10 atendida, contratos compartilhados, zero `any`
Clean code	15	Camadas, nomes, coesão, tratamento de erros, lint limpo
API REST e documentação	10	Rotas, status codes, Problem Details, OpenAPI completo e executável
Banco e migrations	10	Modelagem, constraints, histórico de migrations, persistência
Interface e responsividade	10	Fluxos completos, estados de tela, celular, acessibilidade
Containerização e start.sh	5	Dockerfiles, Compose, healthchecks e robustez do script

Escala

Cada critério recebe nota de 0 a 4, convertida em pontos pelo peso (nota ÷ 4 × peso):

- **0:** ausente.
- **1:** insuficiente, com falhas graves.
- **2:** parcial.
- **3:** atende ao documento.
- **4:** vai além, com qualidade.

Métricas do fluxo de trabalho

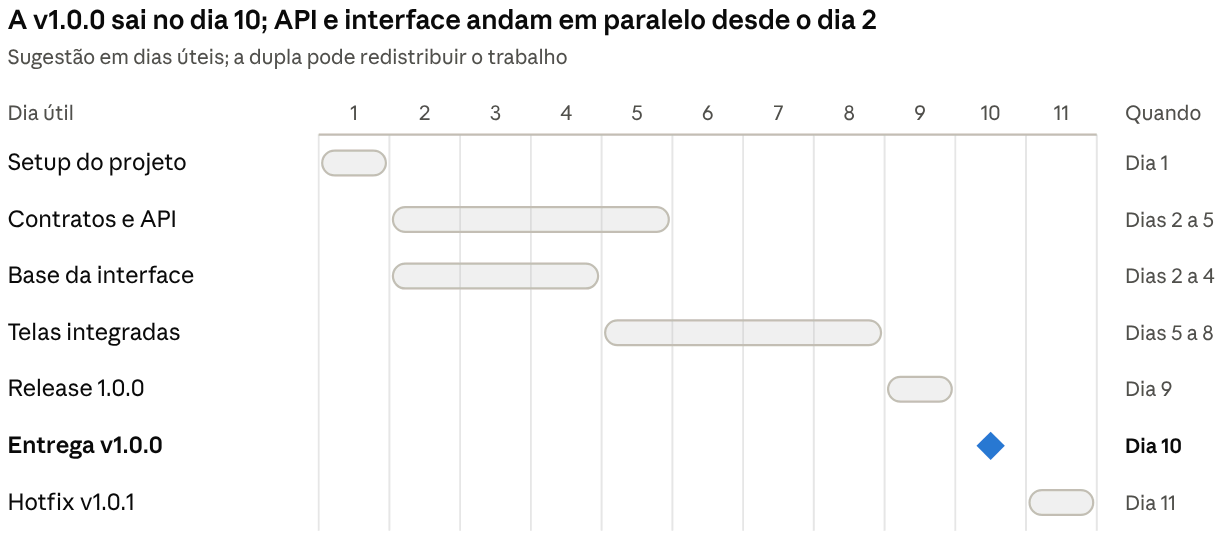
Coletadas no GitHub ao final, para apoiar a nota de fluxo de trabalho e a conversa da apresentação:

- Quantidade de PRs e tamanho médio (linhas alteradas).
- Tempo até a primeira revisão e até o merge.
- Comentários de revisão por PR, e quantos geraram mudança.

- Percentual de commits no padrão Conventional Commits.
- Issues fechadas por PR vinculado.
- Divisão das contribuições entre os dois devs, na API e na interface.
- Tempo do hotfix, da abertura da issue até a tag `v1.0.1`.

17. Cronograma sugerido

São 10 dias úteis até a v1.0.0, com API e interface em paralelo a partir do dia 2, e o hotfix no dia útil seguinte à entrega.



cronograma · 10 dias úteis até a v1.0.0, mais 1 de hotfix

A interface começa pela base (layout, componentes e cliente HTTP) enquanto a API fica pronta; as telas se integram à API a partir do dia 5.

18. Checklist de aceite final

A entrega está pronta quando todos os itens abaixo estiverem marcados.

- [ ] `git clone` seguido de `./start.sh` sobe tudo, sem passos manuais
- [ ] Listagem, wizard, edição, lixeira, soft delete, restauração e exclusão definitiva funcionando
- [ ] Interface utilizável com 360 px de largura
- [ ] Dados continuam lá depois de `docker compose down` e um novo `./start.sh`
- [ ] Migrations versionadas e aplicadas automaticamente na subida

- [ ] Todos os endpoints executáveis em `/docs`
- [ ] Typecheck e lint sem erros em todos os pacotes
- [ ] Nenhum `any` no código
- [ ] Casos de teste dos validadores (seção 7) passando
- [ ] Tags `v1.0.0` e `v1.0.1` na `main`, com CHANGELOG
- [ ] Todo merge em `develop` e `main` feito por PR aprovado
- [ ] README com instruções, arquitetura e decisões